

Báo cáo bài tập lớn: Hệ thống đấu giá trực tuyến AuctionUET

Môn Lập trình Nâng cao – Trường Đại học Công nghệ, ĐHQGHN

Giảng viên: Phạm Bảo Phúc, Trần Ngọc Trúc Linh | Công nghệ: Java 21, Maven, JavaFX, Socket JSON

1. Mục tiêu và phạm vi thực hiện

AuctionUET được xây dựng nhằm mô phỏng một nền tảng đấu giá trực tuyến kiểu eBay Auctions: người bán đăng sản phẩm, người mua cạnh tranh giá trong một khoảng thời gian, hệ thống tự xác định người thắng và xử lý thanh toán. Phạm vi triển khai gồm ứng dụng JavaFX cho người dùng, server socket đa luồng, giao thức JSON dùng chung, lưu trữ JSON, kiểm thử JUnit và CI cơ bản.

Hệ thống tập trung vào các yêu cầu trong đề bài: quản lý người dùng theo vai trò Bidder/Seller/Admin, quản lý sản phẩm và phiên đấu giá, đặt giá hợp lệ, kết thúc phiên tự động, xử lý lỗi, giao diện đồ họa, realtime update, an toàn khi nhiều người đặt giá đồng thời. Nhóm cũng triển khai các phần nâng cao: ví điện tử và đặt cọc 10%, auto-bidding, anti-sniping, biểu đồ giá realtime, dashboard quản trị và voice bidding qua Gemini.

2. Kiến trúc tổng thể

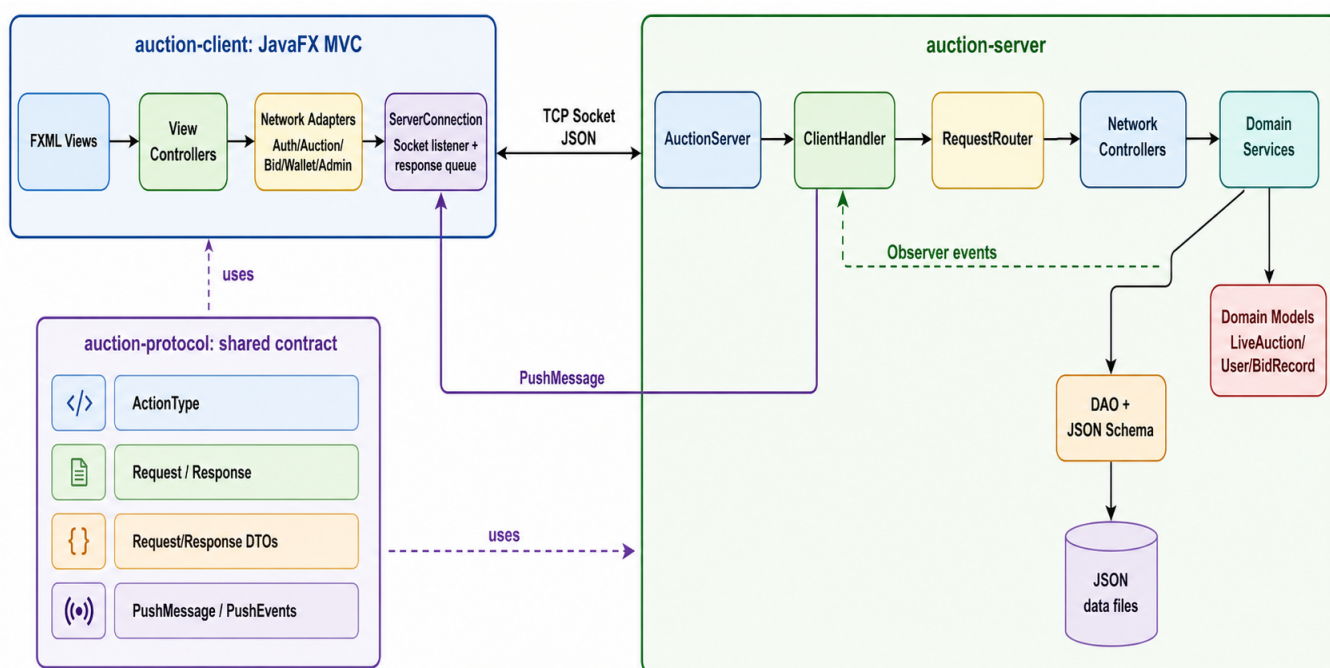


Figure 1: Kiến trúc Client-Server của AuctionUET.

- **Client JavaFX/MVC:** các màn hình FXML và controller nằm trong auction-client; controller chỉ điều phối UI, gọi các adapter mạng như AuthClient, AuctionClient, BidClient, WalletClient. Các lệnh mạng chạy ở background thread và cập nhật UI qua Platform.runLater.
- **Protocol contract-first:** module auction-protocol là nguồn sự thật chung cho ActionType, Request, Response, PushMessage, DTO request/response và enum nghiệp vụ. Cách này tránh tạo DTO lệch nhau giữa client và server.
- **Server phân tầng:** socket được xử lý bởi AuctionServer/ClientHandler; RequestRouter định tuyến tới controller; controller xác thực token/quyền rồi gọi service; service chứa nghiệp vụ và ghi qua DAO/schema JSON.
- **Realtime push:** client subscribe vào phiên; LiveAuction phát sự kiện qua Observer; AuctionManager chuyển tiếp tới các ClientHandler; server gửi PushMessage type PUSH; ServerConnection tách push khỏi response thường.

3. Chức năng đạt được theo barem và bằng chứng

Thiết kế lớp và cây kế thừa

Mốc barem	Tự chấm điểm	Hướng giải quyết và lý do lựa chọn	Bảng chứng từ triển khai
Các lớp chính: <code>User</code> , <code>Bidder/Seller/Admin</code> , <code>Item</code> , <code>Auction</code> , <code>BidTransaction</code>	0.5 / 0.5	Tách runtime domain và persistence schema: user runtime phục vụ phân quyền; sản phẩm/phần/bid/giao dịch lưu bằng schema JSON. Cách này tránh đưa mật khẩu/hash vào domain trả về client.	<code>User</code> , <code>Bidder</code> , <code>Seller</code> , <code>Admin</code> ; <code>ItemSchema/ItemDTO</code> ; <code>AuctionSchema/AuctionDTO</code> ; <code>BidRecord/BidSchema</code> ; <code>TransactionSchema/TransactionDTO</code> .
OOP: <code>Encapsulation</code> , <code>Inheritance</code> , <code>Polymorphism</code> , <code>Abstraction</code>	1 / 1	Dùng abstract class cho vai trò chung, override quyền theo từng role, field private/final và getter để bảo vệ state; <code>LiveAuction</code> đóng gói toàn bộ trạng thái phiên đang chạy.	<code>User</code> có abstract <code>hasPermission</code> và <code>getDisplayName</code> ; <code>Bidder/Seller/Admin</code> override bằng <code>switch</code> ; <code>LiveAuction</code> giữ <code>currentHighestBid</code> , <code>currentWinnerId</code> , <code>bidHistory</code> private.
Design pattern phù hợp	1 / 1	Chọn pattern theo đúng bài toán: Singleton cho manager toàn cục, Observer cho push realtime, DAO cho JSON persistence, MVC cho UI/server, Factory cho Gson adapter.	<code>AuctionManager</code> , <code>DataManager</code> , <code>SessionManager</code> ; <code>AuctionObserver</code> ; <code>UserDAO/AuctionDAO/BidDAO</code> ; <code>GsonFactory</code> ; <code>FXML/controller</code> ở client và <code>controller/service/DAO</code> ở server.

Chức năng chính

Mốc barem	Tự chấm điểm	Hướng giải quyết và lý do lựa chọn	Bảng chứng từ triển khai
Quản lý người dùng, sản phẩm	1 / 1	Người dùng đăng ký/dăng nhập/profile phân quyền theo vai trò; Seller quản lý sản phẩm; Admin phê duyệt/khoá tài khoản. Đặt nghiệp vụ trong service để controller mỏng.	<code>LOGIN</code> , <code>REGISTER</code> , <code>GET_PROFILE</code> ; <code>CREATE_ITEM</code> , <code>GET_MY_ITEMS</code> , <code>DELETE_ITEM</code> ; <code>AuthService</code> , <code>UserService</code> , <code>ItemService</code> , <code>AdminService</code> ; màn <code>Login/Register/MyItems/AdminItems</code> .
Chức năng đấu giá	1 / 1	Server quyết định giá hợp lệ, người dẫn đầu, lịch sử bid và thanh toán; client chỉ gửi request. Cách này chống sửa giá từ UI và giữ dữ liệu nhất quán.	<code>CREATE_AUCTION</code> , <code>START_AUCTION</code> , <code>PLACE_BID</code> , <code>GET_BID_HISTORY</code> , <code>PAY_AUCTION</code> ; <code>AuctionService</code> ; <code>BidService.placeBid</code> ; <code>LiveAuction.placeBid</code> ; <code>BidDAO.save</code> .
Xử lý lỗi và ngoại lệ	1 / 1	Validate DTO ở protocol và service; lỗi nghiệp vụ ném exception rõ nghĩa rồi chuyển về <code>Response.error</code> . Cách này giữ thông báo thống nhất và không lộ stack trace/sensitive data.	<code>ValidatableDTO</code> ; <code>Request.fromDto/getDataAs</code> ; <code>InvalidBidException</code> , <code>AuctionClosedException</code> , <code>InsufficientBalanceException</code> ; <code>GlobalExceptionHandler</code> ; test có case JSON lỗi/token lỗi/quyền lỗi.

Kỹ thuật quan trọng và concurrency

Mốc barem	Tự chấm điểm	Hướng giải quyết và lý do lựa chọn	Bảng chứng từ triển khai
Đấu giá đồng thời an toàn	1 / 1	Giải quyết race condition và lost update: Dùng khóa riêng lẻ từng phiên đấu giá qua <code>ReentrantLock</code> tại <code>LiveAuction</code> để thu hẹp phạm vi lock. Gói kiểm tra nghiệp vụ và trừ tiền cọc vì diện từ trong các phương thức <code>synchronized</code> tại <code>WalletService</code> và <code>BidService</code> . Sử dụng các cấu trúc dữ liệu thread-safe <code>ConcurrentHashMap</code> và <code>CopyOnWriteArrayList</code> cho token và danh sách observer.	<code>BidService.placeBid/setAutoBid</code> <code>synchronized</code> ; <code>WalletService.synchronized</code> ; <code>LiveAuction</code> dùng <code>ReentrantLock</code> ; state dùng <code>ConcurrentHashMap</code> , <code>CopyOnWriteArrayList</code> ; scheduler quản lý start/end.
Realtime update qua Observer/Socket	0.5 / 0.5	Hiện thực Observer Pattern đa tầng: <code>LiveAuction</code> (Subject) gọi <code>notifyObservers</code> đẩy qua Socket dòng tin JSON đơn giản (PushMessage type PUSH). Client dùng luồng nền riêng (daemon thread của <code>ServerConnection</code>) đọc socket, đưa phản hồi thường vào <code>BlockingQueue</code> và chuyển trực tiếp tin push qua UI bằng <code>Platform.runLater</code> .	<code>SUBSCRIBE/UNSUBSCRIBE</code> ; <code>PushActionType</code> , <code>PushEvents</code> , <code>PushMessage</code> ; <code>ClientHandler.sendPush</code> ; <code>ServerConnection</code> tách type PUSH; <code>BiddingController.onPushMessage</code> .

Tích hợp, kiến trúc và chất lượng mã

Mốc barem	Tự chấm điểm	Hướng giải quyết và lý do lựa chọn	Bảng chứng từ triển khai
Kiến trúc Client–Server rõ ràng	0.5 / 0.5	Dùng TCP socket JSON, client không truy cập dữ liệu trực tiếp; mọi nghiệp vụ và persistence nằm ở server.	<code>AuctionServer</code> , <code>ClientHandler</code> , <code>RequestRouter</code> ; <code>ServerConnection</code> ; <code>MessageSerializer</code> ; dữ liệu JSON trong <code>auction-server/data</code> .
MVC: <code>JavaFX+FXML</code> , <code>Controller–Model–DAO</code> server	0.5 / 0.5	Client tách FXML view, controller và network adapter; server tách controller nhận request, service xử lý nghiệp vụ, DAO lưu dữ liệu.	22 FXML trong <code>auction-client</code> ; 22 controller view; server có <code>AuthController</code> , <code>BidController</code> , <code>AuctionController</code> ; service/DAO/schema tương ứng.
Maven/Gradle, convention, mã sạch	0.5 / 0.5	Maven multi-module giúp build contract trước rồi server/client dùng chung; package chia theo layer để tránh DTO/protocol trùng lặp.	Parent pom.xml khai báo 3 module; Java 21; dependency quản lý Gson, SLF4J, Logback, JUnit; source chia protocol, network, domain, persistence.
Unit Test JUnit cho logic quan trọng	0.5 / 0.5	Test tập trung vào service, DAO, auth, auction flow, wallet và integration socket vì đây là các điểm dễ lỗi nghiệp vụ.	Repo có 29 file test Java; chạy cục bộ <code>mvn test --batch-mode</code> pass 148 test, 0 failure/error/skipped; ví dụ <code>AuctionServiceTest</code> , <code>WalletServiceTest</code> , <code>AuthIntegrationTest</code> .
CI/CD cơ bản	0.5 / 0.5	GitHub Actions tự chạy test khi pull request vào main/develop, giảm rủi ro merge code lỗi.	<code>.github/workflows/ci.yml</code> : checkout, setup JDK 21 Temurin, cache Maven, chạy <code>mvn test --batch-mode</code> .

Chức năng nâng cao

Mốc barem	Tự chấm điểm	Hướng giải quyết và lý do lựa chọn	Bảng chứng từ triển khai
Auto-Bidding với maxBid, increment	0.5 / 0.5	Người dùng đặt trần và bước giá qua <code>AutoBidConfig</code> . Chỉ người đang dẫn đầu mới được kích hoạt Auto-Bid nhằm loại bỏ hoàn toàn bidding wars. Khi leader cũ bị vượt mặt, sinh <code>PendingAutoBid</code> có <code>sequenceId</code> . Luồng chạy nền của server sẽ gọi <code>resolvePendingAutoBid</code> dưới khóa <code>ReentrantLock</code> để tự động nâng giá để lên <code>currentHighest + increment</code> (không vượt <code>maxBid</code>).	<code>SetAutoBidRequestDTO; AutoBidConfig; LiveAuction.pendingAutoBid; resolvePendingAutoBid; BidService.schedulePendingAutoBidResponse; CHECK_AUTO_BID.</code>
Anti-sniping khi bid cuối	0.5 / 0.5	Nếu có bid trong cửa sổ <code>antiSnipingWindowSeconds</code> trước khi hết giờ, tự động cộng thêm <code>antiSnipingExtensionSeconds</code> vào <code>endTime</code> của <code>LiveAuction</code> . Server phát sự kiện <code>push AUCTION_EXTENDED</code> , đồng thời hủy bỏ task kết thúc cũ và lập lịch lại một kết thúc mới thông qua <code>ScheduledExecutorService.schedule</code> .	<code>LiveAuction.extendIfSniping; antiSnipingWindowSeconds; antiSnipingExtensionSeconds; push AUCTION_EXTENDED; AuctionManager.extendAuction</code> schedule lại end task.
Bid History Visualization realtime	0.5 / 0.5	Vẽ biểu đồ giá động dạng <code>LineChart</code> của <code>JavaFX</code> . Khi bắt được sự kiện <code>push BID_UPDATE</code> từ server ở <code>background thread</code> , Client bọc lời gọi <code>appendBidPricePoint</code> bên trong <code>Platform.runLater</code> để cập nhật mượt mà các điểm dữ liệu mới (<code>XYChart.Data</code>) lên biểu đồ theo thời gian thực mà không cần tải lại trang.	<code>LineChart<Number, Number></code> trong <code>BiddingController; appendBidPricePoint; loadBidHistory; onPushMessage</code> xử lý <code>BID_UPDATE</code> .
Tính năng sáng tạo khác	0.5 / 0.5	Ví điện tử đóng băng cọc 10% (<code>freezeDeposit</code>), lưu chi tiết qua <code>TransactionSchema</code> phục vụ kiểm toán tài chính. Giám sát hệ thống qua <code>SystemMonitorService</code> . Tích hợp Voice Bidding: ghi âm từ mic, truyền raw audio bất đồng bộ tới Google Gemini API (<code>gemini-3.5-flash</code> kết hợp system prompt) để nhận diện số tiền tiếng Việt thành số thực tự động.	<code>WalletService.holdAuctionDeposit; TransactionService; AdminFinancialController; SystemMonitorService; GeminiVoiceServiceClient; VoiceConfigView.fxml.</code>

4. Phân chia công việc

Thành viên	MSSV	Vai trò	Đóng góp chính
Tạ Hữu Cường	25020053	Team Leader, Core Architect	Phát triển tầng service Auction/Wallet/Bid, refactor sản phẩm sang <code>ItemSchema</code> phẳng, tích hợp Gemini Voice Input/Bidding, kiểm soát concurrency và luồng thanh toán.
Ngô Duy Anh	25020015	Lead Backend, Quality	Thiết kế OOP domain model, hệ thống Admin, integration tests và tài liệu viva/kiểm thử.
Nguyễn Cao Công	25020046	Frontend Lead, Client Dev	Phát triển 22 giao diện <code>JavaFX FXML/CSS</code> , quản lý network connection phía client, Theme Manager và xử lý UI bằng <code>Platform.runLater</code> .
Đào Đình Khánh	25020211	Network, DevOps Lead	Thiết kế protocol <code>Request/Response/Push</code> , socket server và request routing, auto-bidding bảo vệ người dẫn đầu (Pending State), thiết lập GitHub Actions CI.

5. Kết luận

AuctionUET đáp ứng các yêu cầu bắt buộc của đề bài và bổ sung các chức năng nâng cao có bằng chứng trực tiếp trong mã nguồn. Cách triển khai ưu tiên contract-first, phân tầng rõ ràng và xử lý đồng thời ở server, giúp hệ thống vận hành nhất quán khi nhiều client cùng tham gia một phiên đấu giá.